GUIs and Params

Intro to Audio Tech 2
Jocelyn Kavanagh
Thursday, April 9

Georgia Tech

Lesson Outline

- Audio Parameter Value Tree State
- GUIs
- JUCE Plug-in Host

Georgia Tech

Audio Params and Value Tree

Audio Parameter Value Tree State

- A system in JUCE for managing plugin parameters safely and consistently
- Stores all plugin parameters in one place
- Keeps parameters thread-safe (audio thread vs UI thread)
- It allows us to
  - connect seamlessly to our DAW
  - easily save states
  - Keep our parameters organized under one structure
  - quickly build a GUI

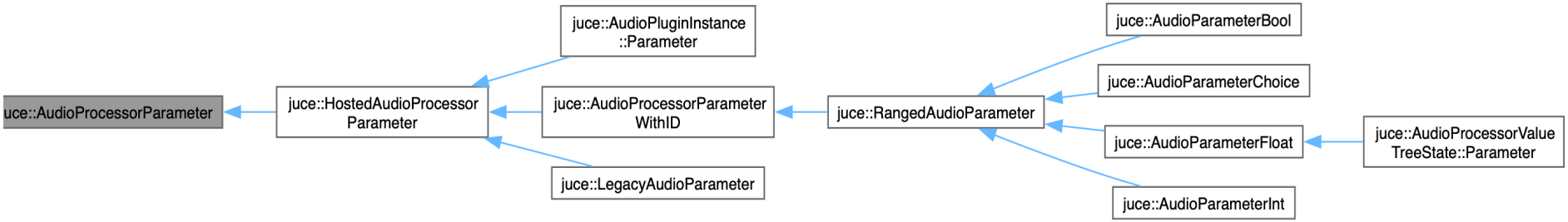
Audio Parameter Value Tree State

AudioProcessorValueTreeState (AudioProcessor &processorToConnectTo,
UndoManager *undoManagerToUse,
const Identifier &valueTreeType, ParameterLayout parameterLayout)

- Arguments:
  - An audio processor - the current one (*this)
  - An undo manager – this controls undo/redo changes from the user
  - An identifier string – just the name of the tree
  - A parameter layout – like a list of all our params -> we have to define this

Parameter Layout and Param Options

- The Parameter Layout object will hold all your audio parameters
- There are 4 different ranged audio parameters
  - AudioParameterBool – for true/false values
  - AudioParameterChoice – for indexed named choices
  - AudioParameterFloat – for float values
  - AudioParameterInt – for int values



Georgia Tech

Notes on creating APVTS and params

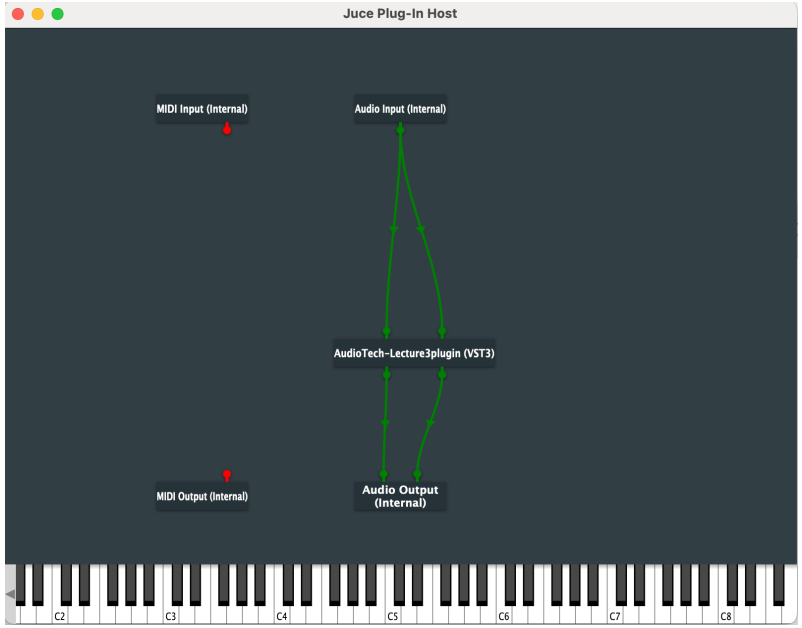
- You can declare the APVTS and the parameter layout in your PluginProcessor header
- You can then initialize your APVTS in your PluginProcessor constructor as part of a member initializer list (just a more efficient way to set up members of a class)
- When we set-up our parameters, we are going to set their type as a unique pointer, so we'll have to use std::makeunique to instantiate

Audio Plug-In Host

Georgia Tech

JUCE Audio Plug-In Host

- To view our plugins, we can load them in a DAW
- However, this sometimes takes a while and is not helpful for debugging/testing
- Instead, we can use the JUCE Audio Plug In Host located in your JUCE folder under extras
- Before you use it, you need to make sure you are building your code to a VST3 or AU (not standalone)



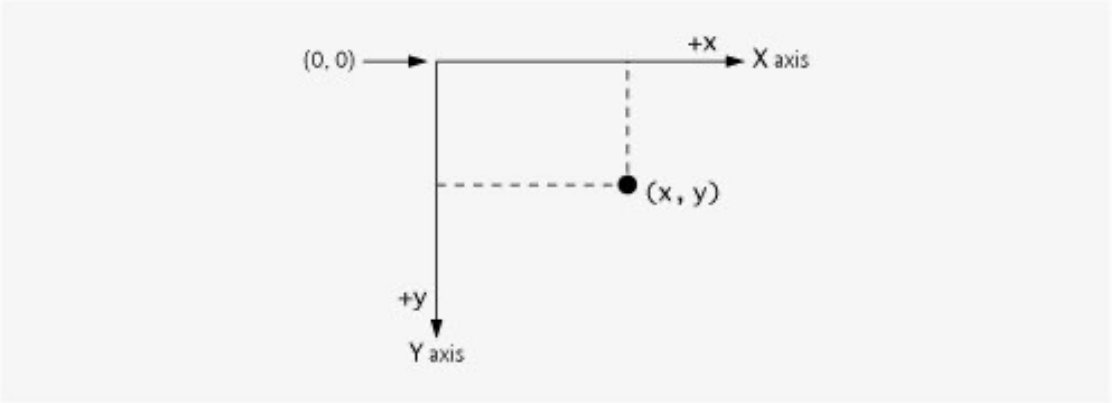
Building a GUI

Plugin Editor

- Our plugin editor (our GUI) is created by the PluginProcessor through hasEditor() and createEditor()
- Visual objects in JUCE are called “components” which all come from the same class (the component class)
  - These include sliders, buttons, shapes, and more
- The component class has a bunch of functions for setting properties of our visual objects
- Our editor is a component which acts as the backdrop or canvas for our GUI
- All additional components that we add will become “children” of the editor
  - We will use addAndMakeVisible() in the editor constructor
- We’ll set our component’s bounds in resized()

Basics of GUI Layout

- The JUCE editor lays out parameters based on a coordinate system where 0,0 is at the top left corner
- Often in GUI design, we use relative positions/sizes to help us layout and center components
  - We'll get more in detail about design later in the course



Attaching Parameters

- After we add our DSP, set-up our params, and layout our components, we need to connect them all together
- We'll use an "attachment" which is a subclass of the APVTS
- We declare the attachment in our editor header and initialize in the editor constructor
- The attachment object keeps the visuals in the GUI and the parameters used in the processBlock() in sync